Machine Learning with Applications 16 (2024) 100546



Contents lists available at ScienceDirect

Machine Learning with Applications

journal homepage: www.elsevier.com/locate/mlwa





A survey of malware detection using deep learning

Ahmed Bensaoud *, Jugal Kalita, Mahmoud Bensaoud

Deptarment of Computer Science, University of Colorado Colorado Springs, CO, USA

ARTICLE INFO

Keywords:
Malware detection
Multi-task learning
Malware image
Generative adversarial networks
Mobile malware
Convolutional neural network

ABSTRACT

The problem of malicious software (malware) detection and classification is a complex task, and there is no perfect approach. There is still a lot of work to be done. Unlike most other research areas, standard benchmarks are difficult to find for malware detection. This paper aims to investigate recent advances in malware detection on MacOS, Windows, iOS, Android, and Linux using deep learning (DL) by investigating DL in text and image classification, the use of pre-trained and multi-task learning models for malware detection approaches to obtain high accuracy and which the best approach if we have a standard benchmark dataset. We discuss the issues and the challenges in malware detection using DL classifiers by reviewing the effectiveness of these DL classifiers and their inability to explain their decisions and actions to DL developers presenting the need to use Explainable Machine Learning (XAI) or Interpretable Machine Learning (IML) programs. Additionally, we discuss the impact of adversarial attacks on deep learning models, negatively affecting their generalization capabilities and resulting in poor performance on unseen data. We believe there is a need to train and test the effectiveness and efficiency of the current state-of-the-art deep learning models on different malware datasets. We examine eight popular DL approaches on various datasets. This survey will help researchers develop a general understanding of malware recognition using deep learning.

1. Introduction

Operating systems such as Windows, Android, Linux, and MacOS are updated every few weeks to protect against critical vulnerabilities. On the other hand, malware authors are also always looking for new ways to finesse their malicious code to overwhelm the new operating system updates. Every operating system is vulnerable. In addition, since operating systems run on desktops and servers, and even on routers, security cameras, drones and other devices, the biggest problem is diversity of systems to protect because all these devices are very different.

Most every day, there is a new story about malicious software in the news. For example, in Oct 2022, cyberattacks coming from a Russia-based hacker group known as Killnet targeted the government services of the state of Colorado, Alabama, Alaska, Delaware, Connecticut, Florida, Mississippi, and Kansas websites.[1] Again in 2022, hackers working on behalf of the Chinese government stole $20 million from covid relief benefits.[2] The increase in the vulnerability of sensitive data due to cyber-attacks, cyber-threats, cyber-crimes, and malware needs to be countered. In 2023, Fig. 1 shows countries that have been attacked by malware and the top origins of these malware.[3]

Researchers have used deep learning to classify malware samples since it generalizes well to unseen data. Our survey focuses on static, dynamic and hybrid malware detection methods in Windows, Android, Linux, MacOS, and iOS. We describe the strengths and weaknesses of deep learning models for malware detection. Most recent research uses deep neural networks (DNNs) for malware classification and achieves high success. State-of-the-art DNN models have been developed against modern malware such as Zeus, Fleeceware, RaaS, Mount Locker, REvil, LockBit, Cryptesla, Snugy, and Shlayer.

The contributions of this paper are as follows:

- It gives the big picture of how hackers attack (Sections 2,3,4,5).
- It presents how to generate images form malware files (Section 6).
- It discusses deep learning models for malware image classification (Section 7).
- It describes feature reduction that can improve performance (Section 8).
- It discusses transfer learning approaches in the classification of malware and what needs to improve for better performance (Section 9).

* Corresponding author.
E-mail addresses: abensaou@uccs.edu (A. Bensaoud), jkalita@uccs.edu (J. Kalita), mbensao2@uccs.edu (M. Bensaoud).

[1] https://www.nbcnews.com/tech/security/colorado-state-websites-struggle-russian-hackers-vow-attack-rcna51012

[2] https://www.nbcnews.com/tech/security/china-hacked-least-six-us-state-governments-report-says-rcna19255

[3] https://attackmap.sonicwall.com/live-attack-map

https://doi.org/10.1016/j.mlwa.2024.100546
Received 24 December 2023; Received in revised form 10 March 2024; Accepted 10 March 2024
Available online 20 March 2024
2666-8270/© 2024 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

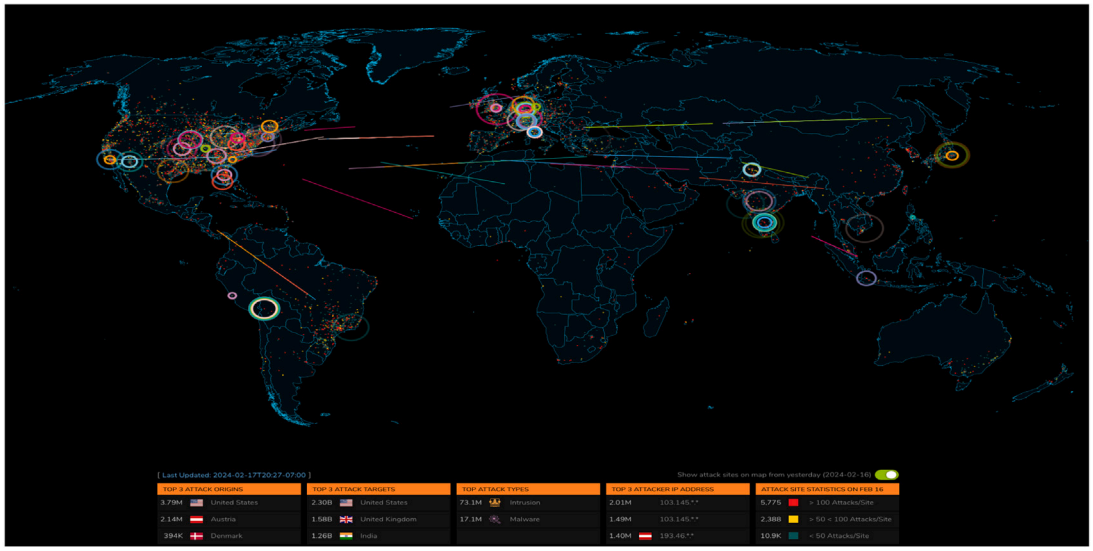


Fig. 1. Worldwide attacks.

- It reviews the use of natural language processing in malware classification (Section 10).
- It presents the deep learning models for cryptographer ransomware (Section 11).
- It shows how we know if we can trust the results of a DL model using Explainable Artificial Intelligence, XAI (Section 12).
- It discusses significant challenge for the reliability and security pozed by adversarial attacks on deep learning models (Section 13).

The rest of this paper, we discuss avenues for future research and we examine the Efficientnet B0, B1, B2, B3, B4, B5, B6, and B7 models on malware images datasets for classification.

2. Mechanics of malware attacks

The hacker has one goal, which is to get malware installed onto a victim's computer. Because most computers are protected by some type of firewall, direct attacks are difficult to impossible to perform. Therefore, attackers attempt to trick the computer into running the malicious code. The most common way to do this is by using documents or executable files. For instance, a hacker may send an email or a phish to the victim with a malicious document attachment or a link to a website where the malicious document is located. Once the victim opens the document, embedded exploits or scripts run and download or extract more malware. This is the real malware the hacker wants to run on the victim's system and is often something like a backdoor or ransomware. However, malicious documents are usually not the final piece of malware in an attack, but are one of the compromised vectors used by the hacker to get on the system. As an example, below we discuss how a PDF document can be used to initiate an attack.

2.1. PDF and document files

When analyzing PDF, we find three things: **Object**, which is the structure of the PDF, **Keywords** which control how the PDF works, and **Data** stored or encoded within a PDF.

- **Objects** are the building blocks of a PDFs. Every PDF starts with a Header which needs to be present in the first 1024 bytes of the documents. Some hackers take advantage of this by putting unrelated data within the first 1024 bytes. This is a very simple technique to try to avoid signature-based detection. PDFs are composed of objects; each section has specific data within the document or performs a specific function. Each object starts with two numbers, followed by the keyword obj, and ends with endobj. There are many kinds of objects, such as font objects, image objects, and even objects that contain metadata.
- There are many **keywords** that begin with a /and describe how the PDF works. Some of the keywords related to malicious activity include */OpenAction*, or its abbreviation */AA*, both of which indicate an automatic action to be performed when the document is viewed.[4] This keyword points to another object that automatically gets opened or executed when the PDF is opened. Malicious PDFs have */OpenAction* pointing to some malicious JavaScript, or an object containing an export; whenever one opens the document, the system is automatically compromised. */JavaScript* or */JS* keyword indicate the presence of JavaScript code. Malicious PDFs usually contain malicious JavaScript to launch an exploit or download additional malware. Some objects can be referred to as */Name* instead of their number. Some PDFs have the ability to have files embedded with keyword */EmbeddedFile*, */URL* or */SubmitForm*. */URL* is accessed or downloaded when the object is loaded.
- PDFs can encode **data** in multiple ways, which is very flexible and can store data in a number of ways. Hackers can encode and hide their data. For example, names are case sensitive, but can be fully or partially hex encoded. More precisely, the # sign followed by two hex characters represents hex encoded data. Data also can be octal encoded or represented by their base eight number. The octal encoded character has a \ followed by three digits between 0 and 7. However, the hackers can mix hex, octal, and ASCII data all together, which makes it possible to hide data such as JavaScript code or URLs.

The names and strings can be encoded, but data streams can be modified and encoded further using filters. **Filters** are algorithms that are applied to the data to encode or compress within the PDF. There are multiple filters that can be used in PDFs, such as */ASCiiHexDecode*, Hex encoding of characters; */LZWDecode*, LZW compression algorithm; */FlateDecode*, Zlib compression; */ASCii85Decode*, ASCII base-85 representation; and */Crypt*, various encryption algorithms. For example, in Fig. 2, we have a PDF document with three objects. Object 1 is a catalog that has OpenAction and is referring to version 0 of object 2, which

[4] https://blog.didierstevens.com/programs/pdf-tools/

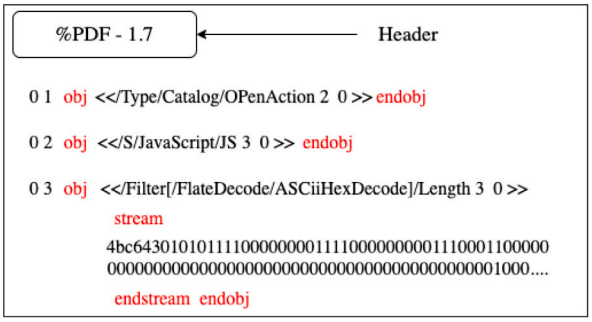


Fig. 2. PDF format example.



Fig. 3. Malicious JavaScript code.

means as soon as the document is opened, Object 2 will be run. Object 2 contains a JavaScript keyword, but we do not see any JavaScript code in this object because the JavaScript keyword refers to another object which is Object 3. Object 3 is a stream object as indicated by the stream keyword and has been ASCiiHex encoded and compressed with the Zlib compression algorithm. However, we have been able to determine that as soon as the PDF opens, JavaScript will be executed, and we do not know what the JavaScript's goal is. If this is a malicious PDF, it can cause problems. In Fig. 3, the JavaScript code references the two hosts' names, performs an HTTP GET request to each, saves an executable file, and finally runs it.

Is malicious JavaScript used only in documents? The answer is everywhere. Malicious JavaScript is used in web pages that are created by web attack kits that perform drive-by downloads. The user opens the website that has been compromised or loads a malicious ad, which then loads malicious JavaScript. Without JavaScript, it is difficult for hackers to get their exploit to work.

Most hackers try to hide what their script is doing using obfuscation techniques. Most techniques used to obfuscate script can be broken down into four different categories. How the **format** of a program is obfuscated is shown in Fig. 4; approaches include adding **extra lines of code**, **obfuscating the data**, and **substituting** variable names.

3. Nature of malware code

The nature of malware code encompasses various characteristics and behaviors that define its purpose and functionality. Malware, short for malicious software, refers to any code or program designed with malicious intent to compromise systems, steal information, or disrupt normal operations. The nature of malware code can vary depending on its specific type and objectives, but some common attributes include:

```
eval(function(p,a,c,k,e,d){e=function(c){return(c<a?'':e(parseInt(c/a
))+((c=c%a)>35?String.fromCharCode(c+29):c.toString(36))};if(!''
.replace(/^/,String)){while(c--){d[e(c)]=k[c]||e(c)}k=[function(e
){return d[e]}];e=function(){return'\\w+'};c=1};while(c--){if(k[c]
){p=p.replace(new RegExp('\\b'+e(c)+'\\b','g'),k[c])}}return p}('3
x="7.v.8 7.t.8".r(" ");3 s=4.5("4.q");3 k=s.p("%h%")+m.l(a)+"g";3
c=4.5("f.e");3 b=4.5("d.u");3 i=0;o(3 j=i;j<x.E;j++){c.9("y","z
://"+x[i]+"/A/?i="+j+&C=D"+n,w);F(c==G){c.9();c.H(k+j+".6",2);c.B
(k+j+".6",1,0)}}',44,44
,'|||var|WScript|CreateObject|exe|hacking|com|open|87|||ADODB|XMLH
TTP|MSXML2|101|TEMP||||fromCharCode|String||for|ExpandEnviromentSt
rings|Shell|split||example2|Stream|example1|false||GET|http|counte
r|RUN|rand|000001|length|if|200|saveTofile'.split('|'),0,{}))
```

Fig. 4. Obfuscated malicious JavaScript code.



Fig. 5. Oops, your files have been encrypted!

3.1. Obfuscation

Obfuscation is an attempt by an author of a piece of code to obscure the meaning, to make something unclear, or make it very difficult to analyze. It may use encryption or compression to hide its true intentions or to evade signature-based detection by security software.

3.2. Payload delivery

Malware code typically carries a payload, which is the malicious action it intends to execute. This can range from stealing sensitive information (e.g., financial data, login credentials) to launching distributed denial-of-service (DDoS) attacks, encrypting files for ransom (ransomware), or providing backdoor access for remote control.

3.3. Command and control (C&C)

Many malware strains establish communication channels with remote servers or command-and-control infrastructure. This allows attackers to remotely control and manage the infected systems, update the malware, and receive stolen data.

3.4. Self-replication

Many malware strains possess the ability to self-replicate, allowing them to spread across networks, devices, or files. This replication can occur through various means, such as attaching to exploiting vulnerabilities, legitimate files, or utilizing network resources.

3.5. Exploitation

Malware leverages vulnerabilities and weaknesses in software, operating systems, or user behavior to gain unauthorized access or control. It can exploit security flaws, network vulnerabilities, or social engineering techniques to compromise systems and execute malicious actions.

3.6. Polymorphism

Some malware utilizes polymorphic or metamorphic techniques to dynamically change its code structure or appearance while preserving its functionality. This makes it more challenging for antivirus software to detect and block.

3.7. Ransomware

A ransomware usually combines cryptography with malware. **How does it work?** The hacker sends the file to an unknowing victim. When the victim opens the file, it executes the malware's payload and encrypts victim data such as photos, documents, multimedia, files, and even confidential records. The hacker often forces the victim to pay in cryptocurrency, in most cases Bitcoin.

Ransomware has worm-like properties and has names such as **WannaCrypt**, **WanaCrypt0r**, **WCRY**, **WanaDecrypt0r**, and **WCrypt**. Each encrypted file is locked by a different key and encrypted with the RSA algorithm, which makes the file unaccessible to the owner who does not have the keys. The WannaCry virus can encrypt a large number of file types. An exhaustive list is given in Appendix A.

The ransomware replaces the desktop wallpaper with the ransom note file by modifying Windows registry. It holds all files hostage to demand ransom payments of $300 and later $600 in the Bitcoin cryptocurrency as shown in Fig. 5.

As an example, on May 11, 2022, Costa Rica's newly elected president had to declare a state of national emergency due to a ransomware attack carried out by the Conti ransomware gang. They requested $10 million, but the demand changed to $20 million after Costa Rica refused to pay.[5] As another example, in October 2022, ransomware gang accessed data on 270,000 patients from Louisiana hospital system.[6]

Understanding the nature of malware code is crucial for developing effective defense mechanisms and mitigating its impact. It enables security professionals to develop robust detection methods, implement security best practices, and respond promptly to evolving threats.

4. Overview & malware detection

Malware detection methods are divided into three types: static, dynamic, and hybrid (Damodaran, Di Troia, Visaggio, Austin, & Stamp, 2017). Static methods inspect an executable file without running it, while dynamic methods must run the executable file and analyze its behaviors inside a controlled environment. In hybrid methods, the information is collected regarding malware from static as well as dynamic analysis.

Some security researchers use static features by decompiling the target file. Naik, Jenkins, Savage, Yang, Boongoen, and Iam-On (2021) proposed a fuzzy-import hashing technique based on static analysis for malware detection. Mohamad, Arif, Ab Razak, Awang, Tuan Mat, Ismail, and Firdaus (2021) proposed machine learning classifiers based on permission-based features for static analysis to detect Android malware.

[5] https://securityintelligence.com/news/costa-rica-state-emergency-ransomware/

[6] https://www.cnn.com/2022/12/28/politics/hackers-access-data-louisiana-hospital-system-ransomware/index.html

Table 1
Syslog and Windows log.

Syslog	Windows logs
IETF standard	Event log
Timestamp	Contains source, event ID, and log level
Standard for network equipment logging	Logs Application, security, network events from a machine or server
Device-ID, severity level, message number, message text	Timestamp, user, computer, and process ID
Can be customized on network equipment for different events and severity levels	Used in most enterprise environments running Windows

Compared to static analysis, dynamic analysis includes system dynamic behavior monitoring, snapshot, debugging, etc. Kim, Suh, Kim, Kim, and Kim (2018) presented a new encoding technique for dynamic features to identify anomalous events using Convolutional Neural Networks (CNNs).

Security researchers have also extracted combined features from different parts of malware files. Bai, Xing, Ma, Li, and Feng (2021) extracted features from static and dynamic analysis of Android apps and applied a deep learning technique. Chaulagain et al. (2020) presented a deep learning-based hybrid analysis technique by collecting different artifacts during static and dynamic analysis to train the deep learning models.

5. Data for malware detection

Numerous system logs of activities of machines such as phones, tablets, laptops, and other devices are generated by the operating system and other infrastructure software. The data are created and stored on the local device and sent to remote servers. Analyzing log data, we can not only detect breaches or suspicious activity, but we can track behavior through the network. Log data allow us to track security events, troubleshoot the infrastructure, and optimize the environment and the machines. Log data can take many different forms like syslog, authentication logs, local security event logs, network asset logs, and system logs. One of goals in malware detection is to be able to read, search, and analyze the data efficiently and effectively.

Table 1 contains some information that is useful from syslog and windows logs. Both kinds of logs have many components in different format that helps us in the investigation.

6. Generating malware images for deep learning

Several tools can visualize and edit a binary file in hexadecimal or ASCII formats such as IDA Pro,[7] x32/x64 Debugger,[8] HxD,[9] PE-bear,[10] Yara,[11] Fiddler,[12] Metadata,[13] XOR analysis,[14] and Embedded strings.[15]

Malware file or code can be used to generate an image by converting the binary, octal, hexadecimal or decimal into a two dimensional matrix of pixels. The image can be grayscale or RGB. In grayscale, pixels are black and white values in the range [0–255] where 0 represents black, and 255 represents white.

Gray image feature: The machine stores images in a matrix of numbers. These numbers, or the pixel values, denote the intensity or brightness of the pixel. Smaller numbers (close to zero) represent black, and larger numbers (closer to 255) denote white (see Fig. 6).

[7] https://hex-rays.com/ida-pro

[8] https://x64dbg.com/#start

[9] https://mh-nexus.de/en/hxd

[10] https://hshrzd.wordpress.com/pe-bear

[11] https://yara.readthedocs.io/en/stable

[12] https://www.telerik.com/purchase/fiddler

[13] https://www.malwarebytes.com/glossary/metadata

[14] https://eternal-todo.com/var/scripts/xorbruteforcer

[15] https://virustotal.github.io/yara/

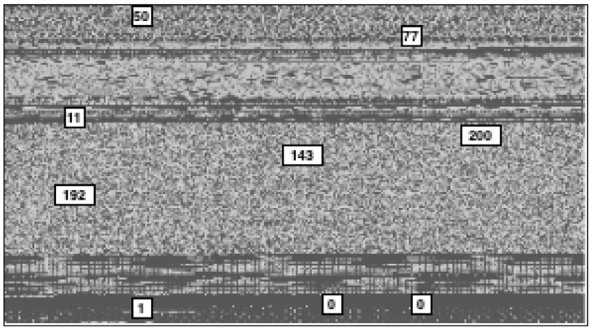


Fig. 6. Malware feature representation in grayscale image. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

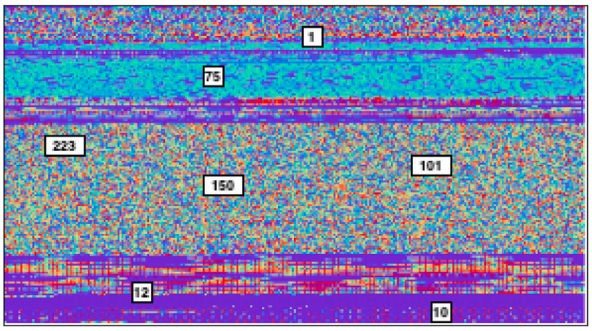


Fig. 7. Malware feature representation in RGB image. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

RGB images: There are three matrices or channels (Red, Green, Blue), where each matrix has values between 0−255. These three colors are combined together in various ways to represent one of 16,777,216 possible colors (see Fig. 7).

Malware can be converted to images in different ways. Yuan, Wang, Liu, Guo, Wu, and Bao (2020) converted malware binaries into Markov images by computing transfer probability of bytes where each pixel is generated by Eq. (1):

$$p_{m,n} = P(n|m) = \frac{f(m,n)}{\sum_{n=0}^{255} f(m,n)} \quad m,n \in \{0,1,\ldots,255\}. \tag{1}$$

Mohammed, Nataraj, Chikkagoudar, Chandrasekaran, and Manjunath (2021) used a vector of 16-bit signed hexadecimal numbers to represent a 256 × 256 image. Then, they computed bi-gram frequency counts which they used as pixel intensity values. Full-frame Discrete Cosine Transform (DCT) (Khayam, 2003) was computed to de-sparsify, and the bigram-DCT was used to represent the output image. Euh, Lee, Kim, and Hwang (2020) proposed Window Entropy Map (WEM) to visualize malware as an image. They calculated the entropy for each byte to measure the degree of uncertainty. Ni, Qian, and Zhang (2018) converted malware code into gray images using SimHash (Charikar, 2002) and then encoded them. They mapped SimHash values to pixels and then converted them to grayscale images.

7. Image classification for malware detection

Deep learning can solve diverse "vision" problems, including malware image classification tasks. Deep learning can extract features automatically obviating manual feature extraction. The content of the malware executable file is first converted into a digital image. Nataraj, Karthikeyan, Jacob, and Manjunath (2011) visualized the byte codes of samples from 25 malware families as grayscale images. Several visualization techniques have been used for malware classification. The basic idea used in these methods is to explore the distinguishing patterns in malware images. In addition, the visualization techniques help find the correlations among different malware families. Some existing approaches generate grayscale images and others generate RGB images. Most existing approaches use global features to generate malware image.

Yuan et al. (2020) proposed a method based on Markov images according to the byte transmission probability matrix. They used a CNN to classify Markov malware images without scaling. Narayanan and Davuluru (2020) proposed an ensemble approach using RNN and CNN architectures for malware image classification. Images were generated from assembly compiled files and classified using CNNs. Zhu, Jang-Jaccard, Singh, Watters, and Camtepe (2021) proposed a Task-Aware Meta Learning-based Siamese Neural Network to classify obfuscated malware images. Their model showed high effectiveness on unique malware signature detection to classify obfuscated malware. Chauhan, Singh, Hooda, and Gupta (2022) visualized malware files in different color modes, RGB, HSV, grayscale, and BGR. They used a support vector machine (SVM) to classify these malware images, with accuracy of 96% in all modes. Darem, Abawajy, Makkar, Alhashmi, and Alanazi (2021) designed a semi-supervised method based on malware image and feature engineering for obfuscated malware detection. The model achieved 99.12% accuracy on obfuscated malware detection. Asam et al. (2021) proposed two malware image classification approaches called Deep Feature Space-based Malware classification (DFS-MC) and Deep Boosted Feature Space-based Malware classification (DBFS-MC). The approach achieved a good accuracy of 98.61% on the MalImg malware dataset.

Xiao, Guo, Shen, Cui, and Jiang (2021) presented a visualization method called Colored Label boxes (CoLab) to specify each section in a PE file and convert it to malware image. The authors built a composed CoLab image, and used VGG16, and Support vector machine for classification. The model was applied on two datasets, VX-Heaven[16] and BIG-2015, with 96.59% and 98.94% average accuracies, respectively. A comparison of reviewed malware images classification is discussed in Table 2.

8. Feature reduction for efficient malware detection

Feature Reduction reduces the number of variables or features in the representation of a data example. Approaches to feature reduction can be divided into two subcategories called (a) Feature Selection which includes methods such as Wrappers, Filters, and Embedded, and (b) Feature Extraction, which includes methods such as Principal Components Analysis (Barath, Ouboti, & Temesguen, 2016). **How does Feature Reduction improve performance?** It does by reducing the number of features that are considered for analysis.

In feature extraction, we start with n features $x_1, x_2, x_3, \ldots., x_n$, which we map to a lower dimensional space to get the new features $z_1, z_2, z_3, \ldots., z_m$ where $m < n$. Each of the new features is usually linear a combination of the original feature set $x_1, x_2, x_3, \ldots., x_n$. Thus, each new feature is obtained as a function F(X) of the original feature set X. This makes a projection of a higher dimensional feature space to a lower dimensional feature space, so that the smaller dimensional feature set may lead to better classification or faster classification (see Eq. (2)).

$$\left[z_1 \ldots z_m\right]^{\intercal} = F\left(\left[x_1 \ldots x_n\right]^{\intercal}\right) \tag{2}$$

In feature selection, we choose a subset of the features, in contrast to feature extraction where we map the original features to a lower

[16] https://archive.org/download/vxheavens-2010-05-18

Table 2
Comparative performance summary of Transfer Learning models for malware image classification.

Reference	Features	Model	Files	Accuracy	Dataset
Çayır, Ünal, and Dağ (2021)	gray-scale images	CapsNet	PE	98.63%	Malimg
Çayır et al. (2021)	gray-scale images	RCNF	PE	98.72%	Malimg
Go et al. (2020)	gray-scale images	ResNeXt	PE	98.32%	Malimg
Bensaoud, Abudawaood, and Kalita (2020)	gray-scale images	Inception V3	PE	99.24%	Malimg
El-Shafai, Almomani, and AlKhayer (2021)	gray-scale images	VGG16	PE	99.97%	Malimg
Hemalatha, Roseline, Geetha, Kadry, and Damaševičius (2021)	gray-scale images	DenseNet	PE	98.23%	Malimg
Hemalatha et al. (2021)	gray-scale images	DenseNet	PE	98.46%	BIG 2015
Lo, Yang, and Wang (2019)	gray-scale images	Xception	PE	99.03%	Malimg
Lo et al. (2019)	gray-scale images	Xception	PE	99.17%	BIG 2015

dimensional space. The smaller dimensional feature set can help produce better as well as faster classification. To do that, we need to find a projection matrix $W \ni \tilde{Z} = W^T \tilde{X}$. We expect from such a projection that the new features are uncorrelated and cannot be reduced further and are non redundant. Next, we need features to have large variance: Why? Because if a feature takes similar values for all the instances, that feature cannot be used as a discriminator.

Feature extraction methods such as a Principal Component Analysis (PCA) (Barath et al., 2016), GIST (Oliva & Torralba, 2001), Hu Moments (Hu, 1962), Color Histogram (Swain & Ballard, 1991), Haralick texture (Lin, Hays, Wu, Kwatra, & Liu, 2004), Discrete Wavelet Transform (DWT) (Kancherla, Donahue, & Mukkamala, 2016), Independent Component Analysis (ICA) (Herault & Jutten, 1986), Linear discriminant analysis (LDA) (Fan, Xu, & Zhang, 2011), Oriented Fast and Rotated BRIEF (ORB) (Rublee, Rabaud, Konolige, & Bradski, 2011), Speeded Up Robust Feature (SURF) (Bay, Tuytelaars, & Van Gool, 2006), Scale Invariant Feature Transform (SIFT) (Lowe, 1999), Dense Scale Invariant Feature Transform (D-SIFT) (Lowe, 1999), Local Binary Patterns (LBPs) (Ojala, Pietikäinen, & Harwood, 1996), KAZE (Alcantarilla, Bartoli, & Davison, 2012) have been combined with machine learning including deep learning. These methods successfully filter the characteristics of malware files. Azad, Riaz, Aftab, Rizvi, Arshad, and Atlam (2022) proposed a method named DEEPSEL (Deep Feature Selection) to identify malicious codes of 39 unique malware families. Their model achieved an accuracy of 83.6% and an F-measure of 82.5%. Tobiyama, Yamaguchi, Shimada, Ikuse, and Yagi (2016) proposed feature extraction based on system calls. Recurrent Neural Network was used to extract features and Convolutional Neural Network to classify these features.

9. Deep transfer leaning models for malware detection

Transfer learning takes place if we have a source model which has some pre-trained knowledge and this knowledge is needed as the foundation to build a new model (Ye & Dai, 2021). For example, using a very large pre-trained convolutional neural network usually involves saving a network that was previously trained on some large dataset, typically on a large-scale image classification task, using a dataset like ImageNet (Russakovsky et al., 2015). After training a network on the ImageNet dataset, we can re-purpose this trained network. Research papers have discussed applying these pre-trained networks to malware image datasets (Bhodia, Prajapati, Di Troia, & Stamp, 2019; Qiao, Zhang, & Zhang, 2020; Rezende, Ruppert, Carvalho, Ramos, & De Geus, 2017; Vasan et al., 2020) that are generated form PE and APK malware files, which are quite different from each other.

Malware image datasets are very different from ImageNet, which is normally used to pre-train the model. The ImageNet dataset and a malware image dataset represent visually completely different images. However, pre-trained still seems to help. Training a machine learning algorithm on large datasets can be done in two ways, as discussed below.

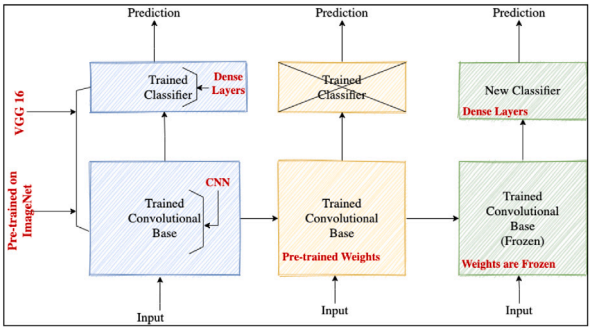


Fig. 8. Feature Extraction for Transfer Learning.

9.1. Using feature extraction

Feature extraction discussed earlier is a practical and common, and low resource-intensive way of using pre-trained networks. It takes the convolutional base of a previously trained network and runs the malware data through it, and then trains a new classifier on top of the output. As shown in Fig. 8, we can choose a network such as VGG16 (Simonyan & Zisserman, 2014) that has been trained on ImageNet, as an example. The input fed at the bottom, goes up to the trained convolutional base, representing the CNN region of the VGG16. The trained classifier resides in the dense region and the prediction is made by this dense region at the end. Usually, we have 1000 neurons at the end to predict the actual ImageNet classes. We take this ImageNet trained model as base, and remove the classifier layer, keeping the convolutional layers of the pre-trained model, along with their weights. In the next step, we attach a new classifier that has new dense layers for malware classification on top. The weights of the base are frozen, which means that the malware input passes through convolutional layers which have their prior weights, during training. However, all dense layers are randomly initialized, and the interconnection weights for these layers are learned during the new training process for detecting malware.

Why remove the original dense layers? What has been observed is that the representations learned by the convolutional base are generic and therefore reusable for a variety of tasks.

9.2. Using fine tuning

Fine-tuning involves changing some of the convolutional layers by learning new weights. In Fig. 9, we have a network divided into three regions. The yellow region is a pre-trained model. The green region represents our dense layers for which we need to learn the weights. During training using a library such as Keras (Ketkar & Santana, 2017) and Tensorflow (Abadi et al., 2016), we can select certain layers and freeze the weights of those layers.

For example, we can select convolutional block one and then freeze all the weights of the convolutional layers, in this block only. This means that during training, everything else will change, but the weights

Table 3
Fine-tuned pre-trained models applied on different malware image datasets.

Setting				Average accuracy			Our dataset
Pre-trained model	Samples	Resize image	Epoch	Malimg	Microsoft challenge	Drebin	Accuracy
EffNet B0	30,000	224	200	92.72%	90.45%	87.23%	94.59%
EffNet B1	30,000	240	200	95.64%	93.65%	88.91%	95.89%
EffNet B2	20,000	260	200	93.84%	91.78%	86.82%	94.12%
EffNet B3	15,000	300	400	90.32%	94.19%	89.35%	95.73%
EffNet B4	20,000	380	400	95.63%	96.68%	90.59%	97.98%
EffNet B5	25,000	456	400	80.19%	87.54%	84.23%	94.68%
EffNet B6	40,000	528	400	85.67%	83.82%	85.43%	93.54%
EffNet B7	30,000	600	1000	82.76%	80.76%	90.57%	88.45%
Inception V4	20,000	229	300	95.98%	93.21%	88.93%	96.39%
Xception	20,000	229	200	89.50%	90.84%	84.39%	93.53%
CapsNet	3000	256	100	88.64%	72.69%	78.68%	92.65%

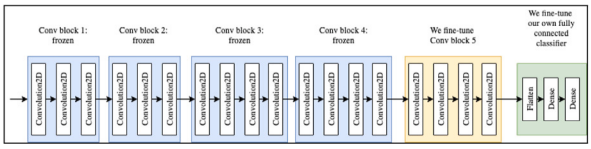


Fig. 9. Fine Tuning of Transfer Learning.

of the convolutional layers in this block will not change. Similarly, we can keep frozen the convolutional layers of the next block as well as blocks three and four if we so wish. Then, we can fine-tune the convolutional layers that are closer to the dense layer. As a result, the initial layers of representation are kept constant, but new representations are learned by later layers (yellow region) as their weights change, evolve and get updated.

Thus, fine-tuning means unfreezing a few of the top layers of a frozen model base used for feature extraction. What we simply do is jointly train the newly added top part of the model (green region) consisting of dense layers, and the top convolutional layers (yellow region), for which we have unfrozen the weights.

Why fine-tune in this manner? Because, we slightly adjust the more abstract representations of the model being reused to make them more relevant for the problem at hand. Sudhakar and Kumar (2021) redesigned ResNet50 (He, Zhang, Ren, & Sun, 2016) by changing the last layer with a fully connected dense layer to detect unknown malware samples without feature engineering. Go et al. (2020) proposed a visualization approach to classify the malware families by using a ResNeXt50 pre-trained model. The model achieved 98.86% accuracy on the Malimg dataset (Nataraj et al., 2011). Çayır et al. (2021) built an ensemble pre-trained capsule network (CapsNet) (Sabour, Frosst, & Hinton, 2017a) based on the bootstrap aggregating approach. The model was trained and tested on two public datasets, Malimg, and BIG2015. Their model achieved F-Score 96.6% on the Malimg dataset (Nataraj et al., 2011) and 98.20% on the BIG2015 dataset.[17] Bensaoud et al. (2020) used six convolutional neural network models for malware classification. Comparison among these models shows that the transfer learning model called Inception-V3 (Szegedy, Vanhoucke, Ioffe, Shlens, & Wojna, 2016) achieved the current state-of-the-art in malware classification. Khan, Zhang, and Kumar (2019) evaluated ResNet and GoogleNet (Szegedy et al., 2015) models for malware detection by converting an APK bytecode into grayscale image. Table 3 summarizes the most transfer learning models for malware classification. We conclude that CNN transfer learning models can be fine-tuned to specific image sizes that are robust enough and accurate to use malware image classification.

Fig. 10 shows how to train the model on an image dataset. We randomly initialize the model, and then train the model on dataset X, which is a large-scale image dataset. This is the pre-training step. Next, we train the model on dataset Y; this dataset is typically smaller than dataset X. This is the fine-tuning step.

State-of-the-art transfer learning models we have trained and evaluated for malware classification are EffNet B0, B1, B2, B3, B4, B5, B6, and B7 (Tan & Le, 2019); Inception-V4 (Szegedy, Ioffe, Vanhoucke, & Alemi, 2017), Xception (Chollet, 2017), and CapsNet (Sabour, Frosst, & Hinton, 2017b) as shown in Table 3. The datasets used are our RGB malware image dataset and two other datasets, namely Malimg Dataset (Nataraj et al., 2011) and Microsoft Malware Dataset (Gibert, Mateu, & Planes, 2020). The accuracy and loss curve plots for EffNet B1, B2, B3, B4, B5, B6, and B7 are shown in Appendix B and EffNet B0 shows in Fig. 11.

We found that the Inception-V4 model is most effective in classifying malware images among the ten models. In addition, the training times for each model increases with increase in the size of input images since the number of network cells grows quickly in GPU RAM.

9.3. Analysis of transfer learning for malware classification

We found that transfer learning based image classification, with a small number of parameters to retrain successfully to classify malware images. On the other hand, we argue that scaled up wider and deeper transfer models with more parameters builds a new model that may improve performance. Inception-V3 and Inception-V4 for malware detection and classification avoid the inefficiencies in classifying unknown malware grayscale and RGB images among transfer learning classification model. There are many transfer learning models techniques such as batch normalization (Kocaman, Shir, & Bäck, 2021), skip connections (Alaraimi, Okedu, Tianfield, Holden, & Uthmani, 2021) that are designed to help in training, but the accuracy still needs to improve. For instance, ResNet-101 and ResNet-50 have similar accuracies in terms of malware detection even though they have very different deep networks (Eum, Lee, & Kwon, 2018).

10. Natural language processing for malware classification

Natural Language Processing (NLP) extracts valuable information so that a program is able to read, understand and derive meaning from human language text or speech. Malware data contain executable files, Microsoft Word files, macro files, logs from different operating systems, emails, network activities, etc. Many of these files contain extensive amounts of text; some others contain snippets of text mixed with code and other information. NLP can be used to enhance malware classification due to the extensive use of text or text-like content within malware. A critical requirement for malware text classification is using effective text representation in the form text encoding. The initial step in text encoding is preprocessing by removing a redundant opcode or API fragments, discarding unnecessary text. After tokenization, there are different types of non-sequential text representations (Jurafsky & Martin, 2021) such as Bag of Words (BoW), Term Frequency Inverse document frequency matrices (TFIDF), Term document matrices

[17] https://www.kaggle.com/c/malware-classification

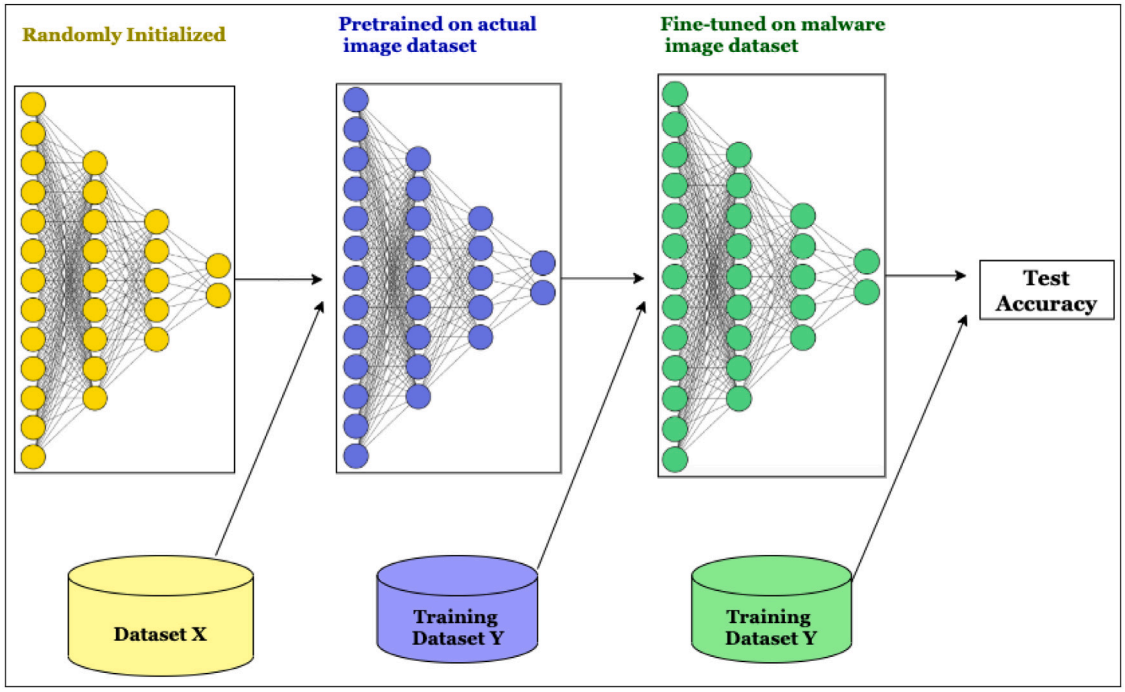


Fig. 10. Transfer Learning steps.

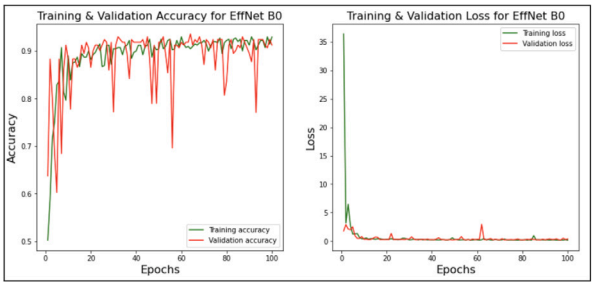


Fig. 11. Training and testing for accuracy and loss of EfficientnetB0.

(TDM), n-grams, One hot encoding, ASCII representations, and modern word embedding such as Word2vec (Mikolov, Chen, Corrado, & Dean, 2013) and Sent2vec (Pagliardini, Gupta, & Jaggi, 2017). Table 4 presents text representation methods used in malware classification. Current word embeddings, when used in malware classification, do not carry much semantic and contextual significance. Bensaoud and Kalita (2024) proposed a novel model for malware classification using API calls and opcodes, incorporating a combined Convolutional Neural Network and Long Short-Term Memory architecture. By transforming features into N-gram sequences and experimenting with various deep learning architectures, including Swin-T and Sequencer2D-L, the method achieves a high accuracy of 99.91%, surpassing state-of-the-art performance. Mimura and Ito (2021) designed NLP-based malware detection by using printable ASCII strings. The model can detect effectively packed malware and anti-debugging. Sequence to Sequence neural models are commonly used for natural languages processing and therefore used for malware detection as well.

10.1. Sequence to sequence neural models

Attention mechanism (Luong, Pham, & Manning, 2015) has achieved high performance in sequential learning applications such as machine translation (Lu et al., 2021), image recognition (Gao, Gong, Ding, & Guo, 2021), text summarization (AlMazrouei, Nelci, Salloum, & Shaalan, 2021), and text classification (Niu, Zhong, & Yu, 2021). Attention mechanism was designed to improve the performance of the encoder–decoder machine translation approach (Ren et al., 2021). The encoder and decoder are usually many stacked RNN layers such as LSTM as shown in Fig. 12. The encoder converts the text into a fixed-length vector while the decoder generates the translation text from this vector. The sequence $\{x_1, x_2, \ldots, x_n\}$ can either be a representation of text or image as shown in Fig. 13. In case of sequences, Recurrent Neural Networks (RNNs) can take two sequences with the same or arbitrary lengths. In Fig. 14, the encoder creates a compressed representation called context vector of the input, while the decoder gets the context vector to generate the output sequence. In this approach, the network is incapable of remembering dependencies in long sentences. This is because the context vector needs to handle potentially long sentences, and a shoot overall representation does not have the especially to store many potential dependencies.

Table 4
The steps of encoding the domain by NLP.

Domain	Notes
www.uccs.edu	Start with domain
uccs	Extract second level
[“u”, “c”, “c”, “s”]	Convert to sequence
[21, 3, 3, 18]	Translate character to numeric values
[0, 0, 0, ….,0 , 21, 3, 3, 18]	Pad sequence

Attention in encoder–decoder: Bahdanau, Cho, and Bengio (2014) proposed an encoder–decoder attention mechanism framework for machine translation. A single fixed context vector is created by an RNN by encoding the input sequence. Rather than using just the fixed vector, we can also use each state of the encoder along with the current decoder state to generate a dynamic context vector. There are two benefits; the first benefit is encoding information contained in a sequence of vectors not just in one single context vector. The second benefit is to choose a subset of these vectors adaptively while decoding the translation.

An attention mechanism is another Lego block that can be used in any deep learning model. Vaswani et al. (2017) showed that an attention mechanism is apparently the only Lego block one needs. It

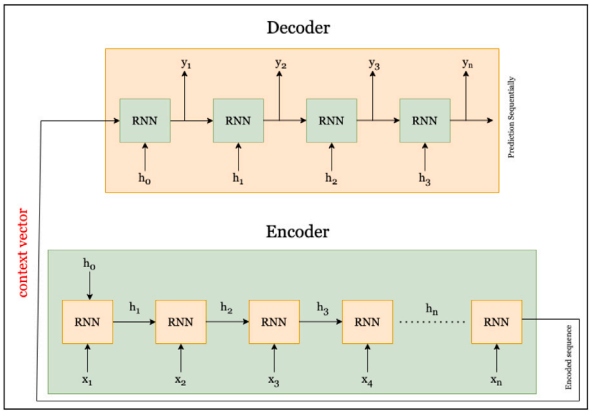


Fig. 12. Encoder and decoder.

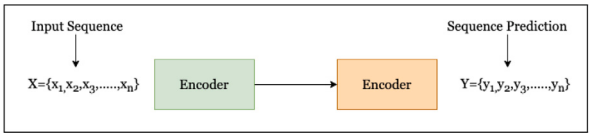


Fig. 13. Encoder and decoder include RNNs.

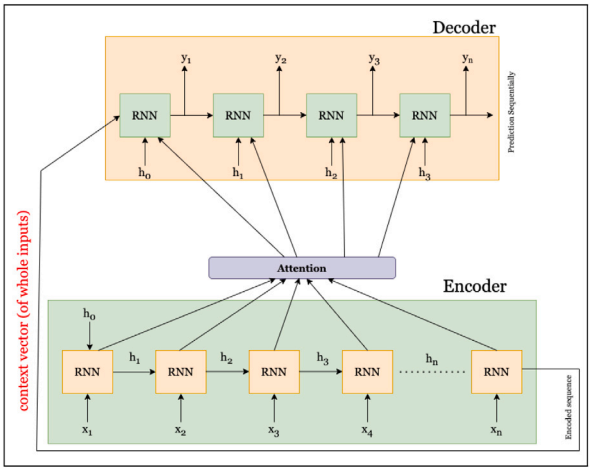


Fig. 14. Encoder and decoder include RNNs with attention mechanism.

improved the performance of a language translation model by dynamically choosing important parts of the input sequence that matter at a certain point in the output sequence. We can entirely replace traditional Recurrent Neural Network (RRN) blocks by an attention mechanism block. When dealing with sequential data, the attention mechanisms allow models to not only perform better but also train faster.

Applying attention mechanism in malware classification: Or-Meir, Cohen, Elovici, Rokach, and Nissim (2021) added an attention mechanism to an LSTM model, which improved accuracy in malware classification. Yakura, Shinozaki, Nishimura, Oyama, and Sakuma (2019) proposed a method by using Convolutional Neural Network with Attention Mechanism for malware image classification. Mimura and Ohminami (2020) proposed a sliding local attention mechanism model (SLAM) based on API execution sequence. Ma et al. (2019) proposed a malware classification framework (ACNN) based on two sections within the malware text, the assembly code and binary code, and converted them into multi-dimensional features. A CNN with attention mechanism for classification has a higher malware image classification accuracy than conventional methods (Yakura et al., 2019). To build

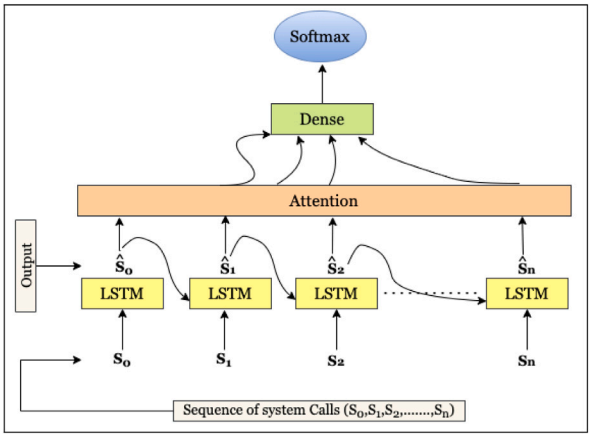


Fig. 15. LSTM with attention mechanism for malware classification.

Table 5
State-of-the-art deep learning models.

Ref	Deep learning approach	OS	Features	Accuracy
Kim, Ban, Ko, Cho, and Yi (2022)	MAPAS	Android	API call graphs	91.27%
Onwuzurike et al. (2019)	MaMaDroid	Android	API calls	84.99%
Kim and Cho (2022)	Deep Generative Model	Android	Dalvik code, API call, Malware images, developers' signature	97.47%
Olani, Wu, Chang, and Shih (2022)	DeepWare	Windows/ Linux	HPC	96.8%
Lian, Nie, Kang, Jia, and Zhang (2022)	Multi-Modal Deep Learning	Windows	Grayscale image, Byte/Entropy Histogram	97.01%
Bensaoud and Kalita (2022)	Deep multi-task learning	Windows Android Linux MacOS	Grayscale color image	99.97%

predictive models using LSTM and attention mechanism for malware classification, we need to add an embedding layer followed by an LSTM layer and dense layers . This approach is superior to capturing a long sequence of Windows API call sequences and using them directly (Girinoto, Setiawan, Putro, & Pramadi, 2020) (see Fig. 15). Malware's longer sequence can be addressed by attention mechanisms that can help detect short repeating patterns and other dependencies (Agrawal, Stokes, Selvaraj, & Marinescu, 2019). While attention mechanism improves accuracy, it suffers from the heavy computation.

Table 5 shows various approaches and their corresponding accuracies. The methods presented, including MAPAS, MaMaDroid, Deep Generative Model, DeepWare, Multi-Modal Deep Learning, and Deep Multi-Task Learning, employ diverse techniques such as API call graph analysis, static analysis, and hybrid deep generative models. Particularly, these methods are evaluated on distinct datasets, indicating that the comparisons are not based on the same dataset. The authors aim to convey the effectiveness of these models in detecting malware across different datasets and scenarios. However, a comprehensive overview of the comparative performance of these methods is needed, highlighting their strengths and capabilities in addressing the challenges of malware detection.

11. Deep learning for cryptographic ransomware

Cryptography has been used traditionally for military and government use, to keep secrets from the enemy. Today most of us use

cryptography when we use commercial websites or services. For example, we use it to protect our emails. A lot of countries try to control the export of cryptography to make sure that good cryptographic algorithms are not in the hands of criminals, enemies, or adversaries. This is the idea behind export administration, and regulations as codified in International Traffic in Arms Regulations (ITAR).[18] In addition, we have various agreements like the Wassenaar Arrangement,[19] where a number of countries got together and developed an agreement for what cryptographic elements can be exported and imported without any type of restrictions. This agreement allows publicly available cryptographic algorithms to be distributed freely. Cryptography provides various security capabilities for us.

- **Confidentiality:** To protect our intellectual property from somebody else being able to get hold of it.
- **Non-repudiation:** To repudiate is to deny. For example, if we use digital signatures, we can provide proof that the message came from the person who signed. We can link the signed document to a trusted person, which gives us trust or assurance in the world of e-contracts and e-commerce. The signer cannot repudiate or deny being the source of the document.
- **Integrity:** Hashing provides integrity, to know that a message was not changed either accidentally or intentionally as it was transmitted or stored. Integrity is built into implementation of electronic communication services today using such as SHA algorithms[20] and MD5.[21]
- **Proof-of-Origin:** Cryptography can be used to prove where a message came from, the idea of Proof-of-Origin.
- **Authenticity:** The idea is to ensure that communication is with the intended person. For example, if we go to a bank's website, then we want to be sure that the website is truly of that bank, not that of an impostor or somebody else masquerading as that bank.

11.1. Operations of cryptography

Cryptographic algorithms come in three basic flavors: Symmetric, Asymmetric, and Hash algorithms. Each of these different types of algorithms serves a different purpose, but all work together in a cryptography system.

Cryptography is a key to keeping communicated information secret by converting it into an unreadable code that is hard to break. To encrypt or encipher is to take a plaintext message and convert it into something unreadable to anyone who does not have a key. To decrypt or decipher is the reverse step.

In Fig. 16, the basic action includes plaintext being fed into a cryptosystem. This process is used to encrypt and decrypt a message. It contains an algorithm that uses a mathematical process to convert a message from plaintext to ciphertext and then back again. The algorithm includes a key or a cryptovariable. The variable is used by the algorithm during the encryption and decryption processes. Typically the key is a secret password, passphrase, or PIN chosen either by the person or by the tool that encrypts the message. This combination of the key (or a cryptovariable) and the algorithm in the cryptosystem produces a unique ciphertext.

In the symmetric algorithm family, a symmetric key is one that is a shared secret between the sender and receiver of the information. The same key used for encryption is also used for decryption. It is not safe to send a copy of the key along with the message that it encrypts.

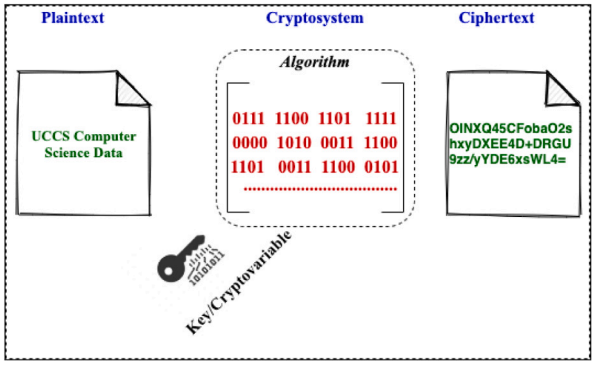


Fig. 16. Crypto Action.

We need to use another mode of communication to transmit the key. For example, Ahmed sends the symmetric key to Bryan using a certain secure node of communication. Once Bryan has the key, Ahmed can encrypt the plaintext message into ciphertext and send it over a public network to Bryan with confidence that it will remain encrypted until Bryan decides to decrypt with the received key.

Multiple attacks, such as a man-in-middle attack, brute force attack, biclique attack, ciphertext only attack, known plaintext attack, chosen plaintext attack, chosen ciphertext attack, and chosen text attack can discover the key to find the plaintext. Attackers know the mathematical relationship of the keys for some algorithms, such as Advanced Encryption Standard (AES) (Heron, 2009), Triple DES (Sasi & Sivanandam, 2015), Blowfish (Mahendra & Prabha, 2022), and Rivest-Shamir-Adleman (RSA) (Kota & Aissi, 2022). We perform cryptanalysts using statistical measures to try to get the cipher type, but a cryptanalyst can only test as many solvers via trial and error to test if the ciphertext was encrypted using a specific cipher. Machine learning can tell us what type a cipher is (Lee, Teh, Jamil, Yan, & Chen, 2021). The cipher type detection problem is classification problem. We can use statistical values as features for machine learning.

11.2. Connection between deep learning and cryptography

A neural network can deal with the complexity of computation applied to perform cryptography. Instead of giving an image to a neural network, we can give ciphertext to the neural network to classify the kind of algorithm that was used to obtain the ciphertext, as shown in Fig. 17. To build a machine learning model, we can represent different features of the cipher, which cryptanalysts usually use to identify them. We need to put an intermediate layer between the network and ciphertext that computes the features, such as Unigram frequencies, Bigram frequencies, Index of Coincidence IoC, HasDoubleLetters, etc., and then we can train the network with millions of ciphertext and all American Cryptogram Association (ACA) cipher types. For example, in Fig. 18, the three blue neural networks are given the frequencies of N-grams (1-grams, 2-grams, 3-grams, 4-grams, etc.), and the green neural network computes HasDoubleLetters. Then we have a hidden layer that connects the input and output layers. Finally, in this case the designed neural network shows 90% Seriated Playfair, and the green neural network shows 10% Bazeries. Baksi (2022) designed a machine-learning model for differential attacks on the non-Markov 8-round GIMLI cipher and GIMLI hash. They applied multi-layer perceptron (MLP), Convolutional Neural Networks (CNN), and Long Short-Term Memory (LSTM).

The ransomware families to encrypt data and force the victim to make payment via cryptocurrency include WannaCry, Locky, Stop, CryptoJoker, CrypoWall, TeslaCrypt, Dharma, Locker, Cerber, and GandCrab. Recently, deep learning algorithms have been used for cryptography (Kok, Azween, & Jhanjhi, 2020). Ding et al. (2020) proposed DeepEDN to fulfill the process of encrypting and decrypting medical

[18] https://csrc.nist.gov/glossary/term/itar

[19] https://www.federalregister.gov/documents/2022/08/15/2022-17125/implementation-of-certain-2021-wassenaar-arrangement-decisions-on-four-section-1758-technologies

[20] https://csrc.nist.gov/glossary/term/sha

[21] https://csrc.nist.gov/glossary/term/md5

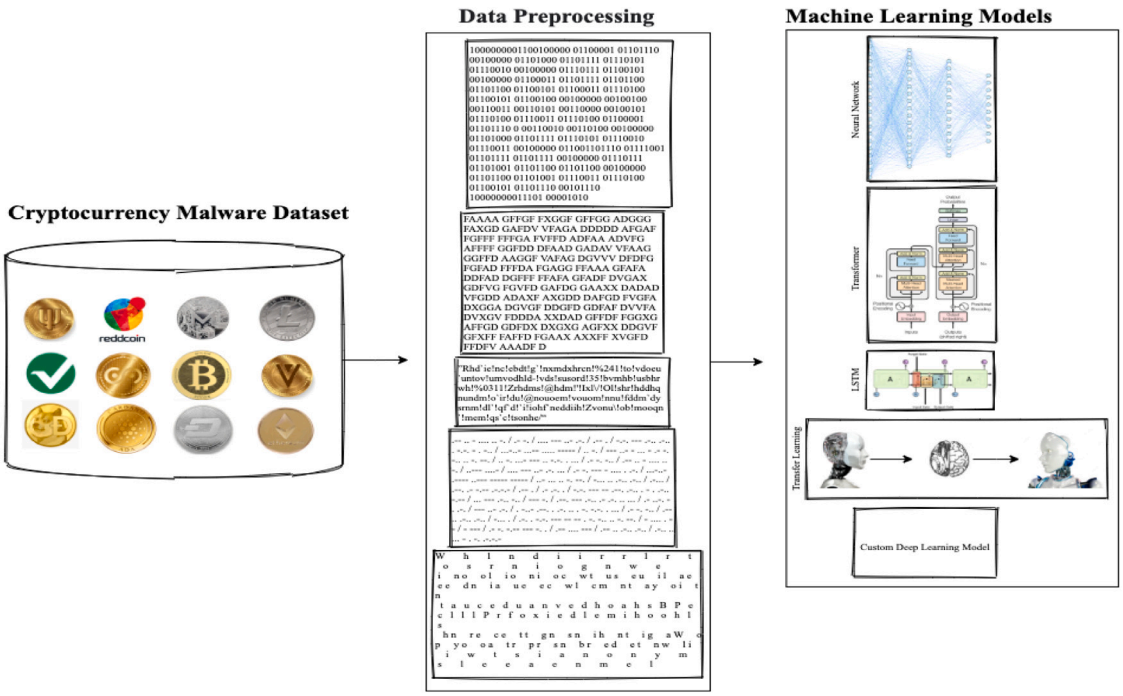


Fig. 17. Cryptocurrency malware detection using machine learning.

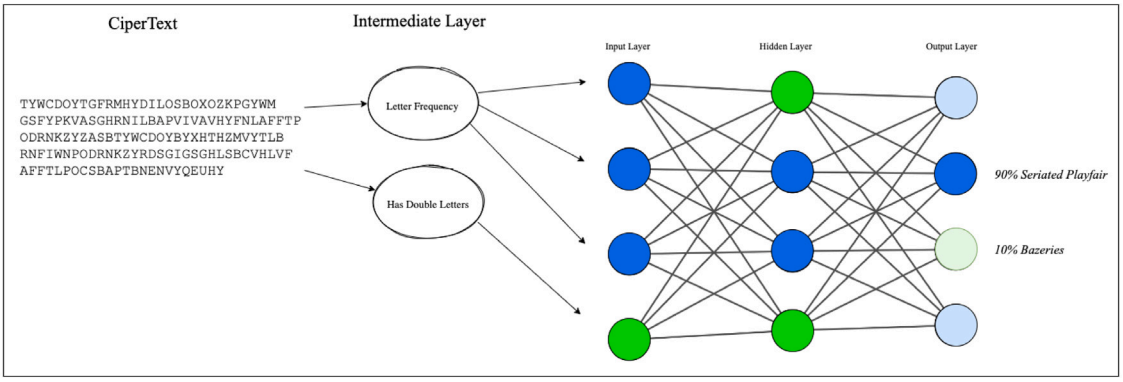


Fig. 18. Detect the Cipher Type With Neural Networks.

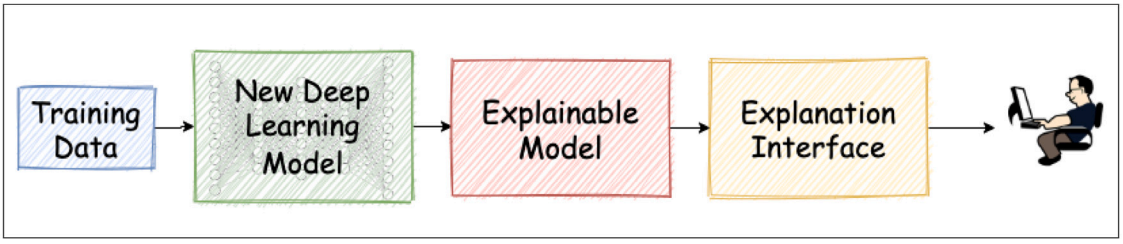


Fig. 19. Using explainable artificial intelligence in deep learning.

images. Kim, Park, Kwon, Jang, and Seo (2021) proposed detection of cryptographic ransomware using Convolutional Neural Network. Their model prevents crypto-ransomware infection by detecting a block cipher algorithm. Sharmeen, Ahmed, Huda, Koçer, and Hassan (2020) proposed an approach to extract the intrinsic attack characteristics of unlabeled ransomware samples using a deep learning-based unsupervised learned model. Fischer et al. (2019) designed a tool to detect security vulnerabilities of cryptographic APIs in Android by achieving an average AUC-ROC of 99.2%.

12. Explainable artificial intelligence (XAI)

Explainable Artificial Intelligence (XAI) is a rapidly emerging field that focuses on creating transparent and interpretable models (see Fig. 19). In the context of malware detection, XAI can help security experts and analysts understand how a machine learning model arrived at its decisions, making it easier to identify and understand false positives and false negatives. By applying XAI techniques, such as Local Interpretable Model-Agnostic Explanations (LIME) (Ribeiro, Singh, & Guestrin, 2016) or Deep Learning Important Features (DeepLIFT)

(Shrikumar, Greenside, & Kundaje, 2017), security teams can gain insights into the most important features and decision-making processes of the model. This can help them identify areas where the model may be vulnerable to evasion or identify new malware strains that the model may have missed. Ultimately, XAI can improve the trustworthiness and reliability of machine learning models for malware detection, enabling more effective threat detection and response.

Nadeem et al. (2022) provided a comprehensive survey and analysis of the current state of research on explainable machine learning (XAI) techniques for computer security applications. The paper highlights the challenges and opportunities for adopting XAI in the security domain and discusses several approaches for designing and evaluating explainable machine learning models. Vivek, Ravi, Mane, and Naidu (2022) proposed an approach for detecting ATM fraud using explainable artificial intelligence (XAI) and causal inference techniques. They presented a detailed analysis of the proposed method and highlighted its effectiveness in improving the accuracy and interpretability of ATM fraud detection systems. Kinkead, Millar, McLaughlin, and O'Kane (2021) proposed an approach that uses LIME to identify important locations in the opcode sequence that are deemed significant by the Convolutional Neural Network (CNN). McLaughlin et al. (2017) used LRP (Bach et al., 2015) and DeepLift (Shrikumar et al., 2017) methods to identify the opcode sequences for most malware families, and they demonstrated that the CNN, while using the DAMD dataset, learned patterns from the underlying op-code representation. Hooker, Erhan, Kindermans, and Kim (2019) proposed a method to remove relevant features detected by an XAI approach and verify the accuracy degradation. Lin, Lee, and Celik (2021) presented seven different XAI methods and automated the evaluation of the correctness of explanation techniques. The first four XAI methods are white-box approaches to determine the importance of input features: Backpropagation (BP), Guided Backpropagation (GBP), Gradient-weighted Class Activation Mapping (GCAM), and Guided GCAM (GGCAM). The last three are black-box approaches that observe an essential feature in the output probability using perturbed samples of the input: Occlusion Sensitivity (OCC), Feature Ablation (FA), and Local Interpretable Model-Agnostic Explanations (LIME).

Guo et al. (2018) proposed an approach called Explaining Deep Learning based Security Applications (LEMNA) for security applications, which generates interpretable features to explain how input samples are classified. Kuppa and Le-Khac (2020) presented a comprehensive analysis of the vulnerability of XAI methods to adversarial attacks in the context of cybersecurity, discussing potential risks associated with deploying XAI models in real-world applications, and proposing a framework for designing robust and secure XAI systems. Rao and Mane (2021) proposed an approach to protect and analyze systems against the alarm-flooding problem using the NSL-KDD dataset. They included a Security Information and Event Management (SIEM) system to generate a zero-shot method for detecting alarm labels specific to adversarial attacks. Although explainable artificial intelligence (XAI) has gained significant attention, its effectiveness in malware detection still requires further investigation to fully comprehend its performance.

13. Adversarial attack on deep neural networks

Adversarial examples refer to maliciously crafted inputs to machine learning models designed to deceive the model into making incorrect predictions. Deep detection in this context refers to the use of deep learning models for detecting and classifying objects or patterns in the input data. Adversarial examples can be specifically crafted to evade deep detection models and cause them to misclassify or miss the target objects or patterns. Therefore, adversarial examples can be seen as a type of attack on deep detection models. Adversarial examples can be generated using a variety of techniques, including optimization-based approaches and perturbation-based approaches, and can be used for various objectives, including evasion attacks and poisoning attacks. Zhong et al. (2023) proposed a novel adversarial malware example generation method called Malfox, which uses conditional generative adversarial networks (conv-GANs) to generate camouflaged adversarial examples against black-box detectors. The presented method was evaluated on two real-world malware detection systems, and the results showed that Malfox achieved high attack success rates while maintaining low detection rates. Zhao et al. (2023) proposed a new method called SAGE for steering the adversarial generation of examples with accelerations. The technique combines the advantages of gradient-based and gradient-free methods to generate more effective and efficient adversarial examples.

The development of defense mechanisms against adversarial attacks is a computationally expensive process, which can potentially affect the performance of the deep learning model. In addition, adversarial examples can impact the generalization ability of deep learning models, resulting in poor performance on new and unseen data. Moreover, generating adversarial examples can be computationally intensive, especially for large datasets and complex models, which can hinder the practical deployment of deep learning models in real-world applications. Thus, further research is required to improve the efficiency and effectiveness of defense mechanisms, as well as the generalization ability and robustness of deep learning models to adversarial attacks.

Hu and Tan (2023) proposed a method to generate adversarial malware examples using Generative Adversarial Networks (GANs) for black-box attacks. Their results show that the generated adversarial malware samples can evade detection by existing machine learning models while maintaining high similarity to the original malware. Ling et al. (2023) conducted a survey of the state-of-the-art in adversarial attacks against Windows PE malware detection, covering various types of attacks and defense mechanisms. The authors also provided insights on potential future research directions in this area. Xu et al. (2023) proposed a semi-black-box adversarial sample attack framework called Ofei that can generate adversarial samples against Android apps deployed on a DLAAS platform. The framework utilizes a multi-objective optimization algorithm to generate robust and stealthy adversarial samples. Qiao et al. (2022) proposed an adversarial detection method for ELF malware using model interpretation and show that their method can effectively identify adversarial ELF malware with high accuracy. The proposed approach combines random forests and LIME to identify the most important features and thus improve the interpretability and robustness of the model. Meenakshi and Maragatham (2023) proposed a defensive technique using Curvelet transform to recognize adversarial iris images, optimizing the image classification accuracy. The designed method was shown to be effective against several existing adversarial attacks on iris recognition systems. Pintor et al. (2022) introduced a method for debugging and improving the optimization of adversarial examples by identifying and analyzing the indicators of attack failure. The proposed method can help to improve the robustness of deep learning models against adversarial attacks.

14. Conclusion

Machine learning has started to gain the attention of malware detection researchers, notably in malware image classification and cipher cryptanalysis. However, more experimentation is required to understand the capabilities and limitations of deep learning when used to detect/classify malware. Deep learning can reduce the need for static and dynamic analysis and discover suspicious patterns. In the future, researchers may consider developing more accurate, robust, scalable, and efficient deep learning models for malware detection systems for various operating systems. Finally, multi-task learning and transfer learning can provide valuable results in classifying all types of malware. Furthermore, we show that the significant challenges of deep learning approaches that need to be considered are hyperparameters optimization, fine-tuning, and size and quality of datasets when features are

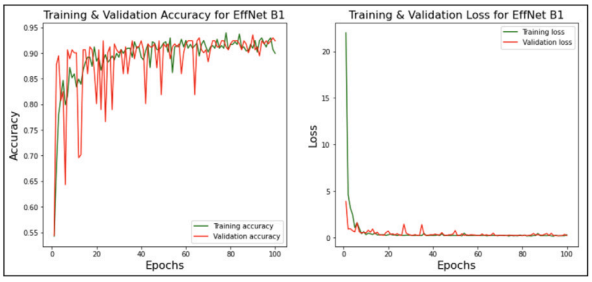


Fig. 20. Training and testing for accuracy and loss of EfficientnetB1.

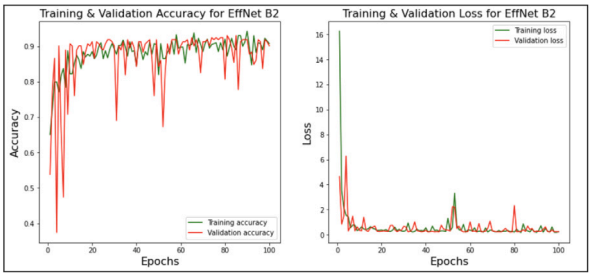


Fig. 21. Training and testing for accuracy and loss of EfficientnetB2.

overweighted or overrepresented. We also illustrate the opportunities and challenges of XAI in deep learning as well as future research directions in the context of malware detection. Finally, we presented the idea of adversarial attacks on deep neural networks by introducing small, carefully crafted perturbations to input data in order to cause misclassification or reduce model performance.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data that has been used is confidential.

Appendix A. File types

. tbk, .jpeg, . brd, .dot, .jpg, .rtf, .doc, .js, .sch, .3dm, .mp3, .sh, .3ds, .key, .sldm, .3g2, .lay, .sldm, .mkv, .std, .asp, .mml, .sti, .avi, .mov, .stw, .backup, . jsp, .suo, .bak, .mp4, .svg, .bat, .mpeg, .swf, .bmp, .mpg, .sxc, .rb, .msg, .sxd, .bz2, .myd, .sxi, .c, .myi, .sxm, .cgm, .nef, .sxw, .class, .odb, .tar, .cmd, .odg, .123, . onetoc2, .odp, .tgz, .crt, .ods, .tif, .3gp, .lay6, .sldx, .7z, .ldf, .slk, .vsd, .m3u, .sln, .aes, .m4u, .snt, .ai, .max, .sql, . ppam, .mdb, .sqlite3, .asc, .mdf, .sqlitedb, .asf, .mid, .stc, .asm, .cs, .odt, .tiff, .csr, .cpp, .txt, .csv, .pas, .vmx, .docb, .pdf, .vob, .docm, .pem, . accdb, .docx, .pfx, .vsdx, .602, . p12, .wav, .dotm, .pl, .wb2, .dotx, .png, .wk1, .dwg, .pot, . xltx, .edb, .potm, .wma, .eml, .potx, .wmv, .fla, .ARC, .xlc, .flv, .pps, .xlm, .frm, .ppsm, .xls, .gif, .ppsx, .xlsb, .gpg, .ppt, .xlsm, .gz, .pptm, .xlsx, .h, .pptx, .xlt, .hwp, .ps1, .xltm, .ibd, .psd, .wks, .iso, .pst, .xlw, .jar, .rar, . djvu, .java, .raw., .ost, .uop, .db, .otg, .uot, .dbf, .otp, .vb, .dch, .ots, .vbs, .der", .ott, .vcd, .dif, .php, .vdi, .dip, .PAQ, .vmdk, .zip

Appendix B. The accuracy and loss curves plots

See Figs. 20–26.

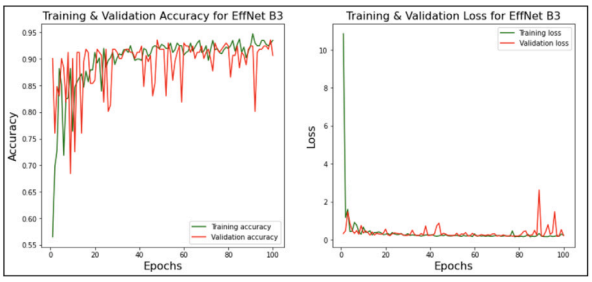


Fig. 22. Training and testing for accuracy and loss of EfficientnetB3.

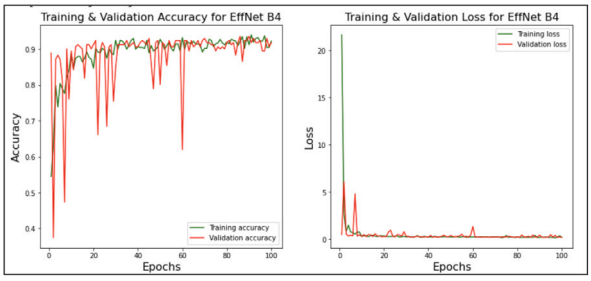


Fig. 23. Training and testing for accuracy and loss of EfficientnetB4.

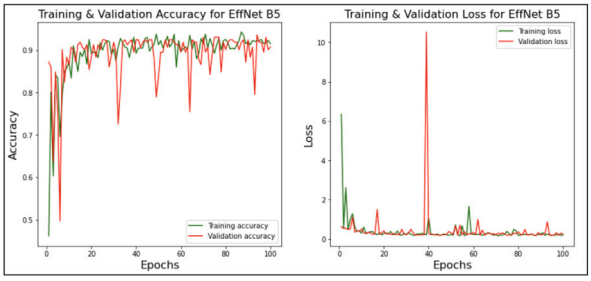


Fig. 24. Training and testing for accuracy and loss of EfficientnetB5.

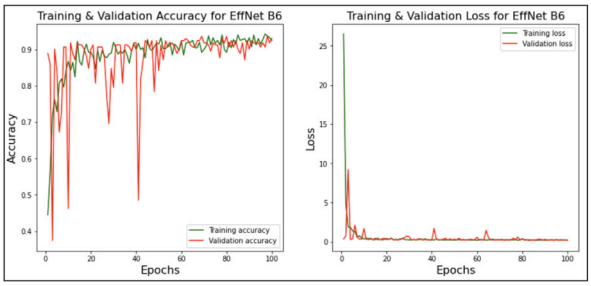


Fig. 25. Training and testing for accuracy and loss of EfficientnetB6.

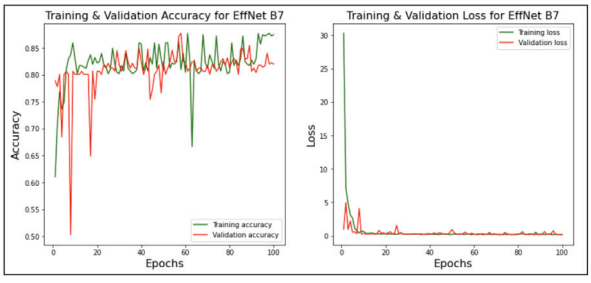


Fig. 26. Training and testing for accuracy and loss of EfficientnetB7.

References

Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., et al. (2016). Tensorflow: A system for large-scale machine learning. In *12th {USeNIX} symposium on operating systems design and implementation* {oSDI} 16, (pp. 265–283).

Agrawal, R., Stokes, J. W., Selvaraj, K., & Marinescu, M. (2019). Attention in recurrent neural networks for ransomware detection. In *ICASSP 2019 - 2019 IEEE international conference on acoustics, speech and signal processing* (pp. 3222–3226).

Alaraimi, S., Okedu, K. E., Tianfield, H., Holden, R., & Uthmani, O. (2021). Transfer learning networks with skip connections for classification of brain tumors. *International Journal of Imaging Systems and Technology*.

Alcantarilla, P. F., Bartoli, A., & Davison, A. J. (2012). KAZE features. In *European conference on computer vision* (pp. 214–227). Springer.

AlMazrouei, R. Z., Nelci, J., Salloum, S. A., & Shaalan, K. (2021). Feasibility of using attention mechanism in abstractive summarization. In *International conference on emerging technologies and intelligent systems* (pp. 13–20). Springer.

Asam, M., Hussain, S. J., Mohatram, M., Khan, S. H., Jamal, T., Zafar, A., et al. (2021). Detection of exceptional malware variants using deep boosted feature spaces and machine learning. *Applied Sciences*, *11*(21), 10464.

Azad, M. A., Riaz, F., Aftab, A., Rizvi, S. K. J., Arshad, J., & Atlam, H. F. (2022). DEEPSEL: A novel feature selection for early identification of malware in mobile applications. *Future Generation Computer Systems*, *129*, 54–63.

Bach, S., Binder, A., Montavon, G., Klauschen, F., Müller, K.-R., & Samek, W. (2015). On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation. *PLoS One*, *10*(7), Article e0130140.

Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural machine translation by jointly learning to align and translate. arXiv preprint arXiv:1409.0473.

Bai, Y., Xing, Z., Ma, D., Li, X., & Feng, Z. (2021). Comparative analysis of feature representations and machine learning methods in Android family classification. *Computer Networks*, *184*, Article 107639.

Baksi, A. (2022). Machine learning-assisted differential distinguishers for lightweight ciphers. In *Classical and physical security of symmetric key cryptographic algorithms* (pp. 141–162). Springer.

Barath, N., Ouboti, D., & Temesguen, M. (2016). Pattern recognition algorithms for malware classification. In *Proceeding of 2016 IEEE conference of aerospace and electronics* (pp. 338–342).

Bay, H., Tuytelaars, T., & Van Gool, L. (2006). Surf: Speeded up robust features. In *European conference on computer vision* (pp. 404–417). Springer.

Bensaoud, A., Abudawaood, N., & Kalita, J. (2020). Classifying malware images with convolutional neural network models. *International Journal of Network Security*, *22*, 1022–1031.

Bensaoud, A., & Kalita, J. (2022). Deep multi-task learning for malware image classification. *Journal of Information Security and Applications*, *64*, Article 103057.

Bensaoud, A., & Kalita, J. (2024). CNN-LSTM and transfer learning models for malware classification based on opcodes and API calls. *Knowledge-Based Systems*, Article 111543.

Bhodia, N., Prajapati, P., Di Troia, F., & Stamp, M. (2019). Transfer learning for image-based malware classification. arXiv preprint arXiv:1903.11551.

Çayır, A., Ünal, U., & Dağ, H. (2021). Random CapsNet forest model for imbalanced malware type classification task. *Computers & Security*, *102*, Article 102133.

Charikar, M. S. (2002). Similarity estimation techniques from rounding algorithms. In *Proceedings of the thiry-fourth annual ACM symposium on theory of computing* (pp. 380–388).

Chauhan, D., Singh, H., Hooda, H., & Gupta, R. (2022). Classification of malware using visualization techniques. In *International conference on innovative computing and communications* (pp. 739–750). Springer.

Chaulagain, D., Poudel, P., Pathak, P., Roy, S., Caragea, D., Liu, G., et al. (2020). Hybrid analysis of android apps for security vetting using deep learning. In *2020 IEEE conference on communications and network security* (pp. 1–9).

Chollet, F. (2017). Xception: Deep learning with depthwise separable convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 1251–1258).

Damodaran, A., Di Troia, F., Visaggio, C. A., Austin, T. H., & Stamp, M. (2017). A comparison of static, dynamic, and hybrid analysis for malware detection. *Journal of Computer Virology and Hacking Techniques*, *13*(1), 1–12.

Darem, A., Abawajy, J., Makkar, A., Alhashmi, A., & Alanazi, S. (2021). Visualization and deep-learning-based malware variant detection using OpCode-level features. *Future Generation Computer Systems*, *125*, 314–323.

Ding, Y., Wu, G., Chen, D., Zhang, N., Gong, L., Cao, M., et al. (2020). DeepEDN: a deep-learning-based image encryption and decryption network for internet of medical things. *IEEE Internet of Things Journal*, *8*(3), 1504–1518.

El-Shafai, W., Almomani, I., & AlKhayer, A. (2021). Visualized malware multi-classification framework using fine-tuned CNN-based transfer learning models. *Applied Sciences*, *11*(14).

Euh, S., Lee, H., Kim, D., & Hwang, D. (2020). Comparative analysis of low-dimensional features and tree-based ensembles for malware detection systems. *IEEE Access*, *8*, 76796–76808.

Eum, S., Lee, H., & Kwon, H. (2018). Going deeper with CNN in malicious crowd event classification. *Vol. 10646*, In *Signal processing, sensor/information fusion, and target recognition XXVII*. International Society for Optics and Photonics, Article 1064616.

Fan, Z., Xu, Y., & Zhang, D. (2011). Local linear discriminant analysis framework using sample neighbors. *IEEE Transactions on Neural Networks*, *22*(7), 1119–1132.

Fischer, F., Xiao, H., Kao, C.-Y., Stachelscheid, Y., Johnson, B., Razar, D., et al. (2019). Stack overflow considered helpful! deep learning security nudges towards stronger cryptography. In *28th {USeNIX} security symposium* {USeNIX} security 19, (pp. 339–356).

Gao, Y., Gong, H., Ding, X., & Guo, B. (2021). Image recognition based on mixed attention mechanism in smart home appliances. *Vol. 5*, In *2021 IEEE 5th advanced information technology, electronic and automation control conference* (pp. 1501–1505).

Gibert, D., Mateu, C., & Planes, J. (2020). The rise of machine learning for detection and classification of malware: Research developments, trends and challenges. *Journal of Network and Computer Applications*, *153*, Article 102526.

Girinoto, Setiawan, H., Putro, P. A. W., & Pramadi, Y. R. (2020). Comparison of LSTM architecture for malware classification. In *2020 international conference on informatics, multimedia, cyber and information system* (pp. 93–97).

Go, J. H., Jan, T., Mohanty, M., Patel, O. P., Puthal, D., & Prasad, M. (2020). Visualization approach for malware classification with ResNeXt. In *2020 IEEE congress on evolutionary computation* (pp. 1–7).

Guo, W., Mu, D., Xu, J., Su, P., Wang, G., & Xing, X. (2018). Lemna: Explaining deep learning based security applications. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security* (pp. 364–379).

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770–778).

Hemalatha, J., Roseline, S. A., Geetha, S., Kadry, S., & Damaševičius, R. (2021). An efficient DenseNet-based deep learning model for malware detection. *Entropy*, *23*(3), 344.

Herault, J., & Jutten, C. (1986). Space or time adaptive signal processing by neural network models. *Vol. 151*, In *AIP conference proceedings* (1), (pp. 206–211). American Institute of Physics.

Heron, S. (2009). Advanced encryption standard (AES). *Netw. Secur.*, *2009*(12), 8–12.

Hooker, S., Erhan, D., Kindermans, P.-J., & Kim, B. (2019). A benchmark for interpretability methods in deep neural networks. In *Advances in neural information processing systems*: *vol. 32*.

Hu, M.-K. (1962). Visual pattern recognition by moment invariants. *IRE Transactions on Information Theory*, *8*(2), 179–187.

Hu, W., & Tan, Y. (2023). Generating adversarial malware examples for black-box attacks based on GAN. In *Data mining and big data: 7th international conference, DMBD 2022, Beijing, China, November 21–24, 2022, proceedings, Part II* (pp. 409–423). Springer.

Jurafsky, D., & Martin, J. (2021). *Speech and Language Processing* (third ed.).

Kancherla, K., Donahue, J., & Mukkamala, S. (2016). Packer identification using Byte plot and Markov plot. *Journal of Computer Virology and Hacking Techniques*, *12*(2), 101–111.

Ketkar, N., & Santana, E. (2017). *Vol. 1*, *Deep learning with Python*. Springer.

Khan, R. U., Zhang, X., & Kumar, R. (2019). Analysis of ResNet and GoogleNet models for malware detection. *Journal of Computer Virology and Hacking Techniques*, *15*(1), 29–37.

Khayam, S. A. (2003). *Vol. 114*, *The discrete cosine transform (DCT): theory and application* (pp. 1–31). Michigan State University, Citeseer.

Kim, J., Ban, Y., Ko, E., Cho, H., & Yi, J. H. (2022). MAPAS: a practical deep learning-based android malware detection system. *International Journal of Information Security*, 1–14.

Kim, J.-Y., & Cho, S.-B. (2022). Obfuscated malware detection using deep generative model based on global/local features. *Computers & Security*, *112*, Article 102501.

Kim, H., Park, J., Kwon, H., Jang, K., & Seo, H. (2021). Convolutional neural network-based cryptography ransomware detection for low-end embedded processors. *Mathematics*, *9*(7), 705.

Kim, T., Suh, S. C., Kim, H., Kim, J., & Kim, J. (2018). An encoding technique for CNN-based network anomaly detection. In *2018 IEEE international conference on big data (big data)* (pp. 2960–2965).

Kinkead, M., Millar, S., McLaughlin, N., & O'Kane, P. (2021). Towards explainable CNNs for android malware detection. *Procedia Computer Science*, *184*, 959–965.

Kocaman, V., Shir, O. M., & Bäck, T. (2021). Improving model accuracy for imbalanced image classification tasks by adding a final batch normalization layer: An empirical study. In *2020 25th international conference on pattern recognition* (pp. 10404–10411).

Kok, S., Azween, A., & Jhanjhi, N. (2020). Evaluation metric for crypto-ransomware detection using machine learning. *Journal of Information Security and Applications*, *55*, Article 102646.

Kota, C. M., & Aissi, C. (2022). Implementation of the RSA algorithm and its cryptanalysis. In *2002 GSW*.

Kuppa, A., & Le-Khac, N.-A. (2020). Black box attacks on explainable artificial intelligence (XAI) methods in cyber security. In *2020 international joint conference on neural networks* (pp. 1–8). IEEE.

Lee, T. R., Teh, J. S., Jamil, N., Yan, J. L. S., & Chen, J. (2021). Lightweight block cipher security evaluation based on machine learning classifiers and active S-boxes. *IEEE Access*, *9*, 134052–134064.

Lian, W., Nie, G., Kang, Y., Jia, B., & Zhang, Y. (2022). Cryptomining malware detection based on edge computing-oriented multi-modal features deep learning. *China Communications*, *19*(2), 174–185.

Lin, W.-C., Hays, J., Wu, C., Kwatra, V., & Liu, Y. (2004). *A comparison study of four texture synthesis algorithms on regular and near-regular textures*: *Tech. Rep.*, Citeseer.

Lin, Y.-S., Lee, W.-C., & Celik, Z. B. (2021). What do you see? Evaluation of explainable artificial intelligence (XAI) interpretability through neural backdoors. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining* (pp. 1027–1035).

Ling, X., Wu, L., Zhang, J., Qu, Z., Deng, W., Chen, X., et al. (2023). Adversarial attacks against Windows PE malware detection: A survey of the state-of-the-art. *Computers & Security*, Article 103134.

Lo, W. W., Yang, X., & Wang, Y. (2019). An Xception convolutional neural network for malware classification with transfer learning. In *2019 10th IFIP international conference on new technologies, mobility and security* (pp. 1–5).

Lowe, D. (1999). Object recognition from local scale-invariant features. *Vol. 2*, In *Proceedings of the seventh IEEE international conference on computer vision* (pp. 1150–1157 vol.2).

Lu, Z., Li, X., Liu, Y., Zhou, C., Cui, J., Wang, B., et al. (2021). Exploring multi-stage information interactions for multi-source neural machine translation. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 1.

Luong, M.-T., Pham, H., & Manning, C. D. (2015). Effective approaches to attention-based neural machine translation. arXiv preprint arXiv:1508.04025.

Ma, X., Guo, S., Li, H., Pan, Z., Qiu, J., Ding, Y., et al. (2019). How to make attention mechanisms more practical in malware classification. *IEEE Access*, *7*, 155270–155280.

Mahendra, M., & Prabha, P. S. (2022). Classification of security levels to enhance the data sharing transmissions using blowfish algorithm in comparison with data encryption standard. In *2022 international conference on sustainable computing and data communication systems* (pp. 1154–1160). IEEE.

McLaughlin, N., Martinez del Rincon, J., Kang, B., Yerima, S., Miller, P., Sezer, S., et al. (2017). Deep android malware detection. In *Proceedings of the seventh ACM on conference on data and application security and privacy* (pp. 301–308).

Meenakshi, K., & Maragatham, G. (2023). An optimised defensive technique to recognize adversarial Iris images using Curvelet transform. *Intelligent Automation & Soft Computing*, *35*(1), 627–643.

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781.

Mimura, M., & Ito, R. (2021). Applying NLP techniques to malware detection in a practical environment. *International Journal of Information Security*, 1–13.

Mimura, M., & Ohminami, T. (2020). Using LSI to detect unknown malicious VBA macros. *Journal of Information Processing*, *28*, 493–501.

Mohamad, Arif, J., Ab Razak, M. F., Awang, S., Tuan Mat, S. R., Ismail, N. S. N., et al. (2021). A static analysis approach for android permission-based malware detection systems. *PLoS One*, *16*(9), Article e0257968.

Mohammed, T. M., Nataraj, L., Chikkagoudar, S., Chandrasekaran, S., & Manjunath, B. (2021). Malware detection using frequency domain-based image visualization and deep learning. arXiv preprint arXiv:2101.10578.

Nadeem, A., Vos, D., Cao, C., Pajola, L., Dieck, S., Baumgartner, R., et al. (2022). Sok: Explainable machine learning for computer security applications. arXiv preprint arXiv:2208.10605.

Naik, N., Jenkins, P., Savage, N., Yang, L., Boongoen, T., & Iam-On, N. (2021). Fuzzy-import hashing: A static analysis technique for malware detection. *Forensic Science International: Digital Investigation*, *Vol. 37*, Article 301139.

Narayanan, B. N., & Davuluru, V. S. P. (2020). Ensemble malware classification system using deep neural networks. *Electronics*, *9*(5), 721.

Nataraj, L., Karthikeyan, S., Jacob, G., & Manjunath, B. S. (2011). Malware images: visualization and automatic classification. In *Proceedings of the 8th international symposium on visualization for cyber security* (pp. 1–7).

Ni, S., Qian, Q., & Zhang, R. (2018). Malware identification using visualization images and deep learning. *Computers & Security*, *77*, 871–885.

Niu, Z., Zhong, G., & Yu, H. (2021). A review on the attention mechanism of deep learning. *Neurocomputing*, *452*, 48–62.

Ojala, T., Pietikäinen, M., & Harwood, D. (1996). A comparative study of texture measures with classification based on featured distributions. *Pattern Recognition*, *29*(1), 51–59.

Olani, G., Wu, C.-F., Chang, Y.-H., & Shih, W.-K. (2022). DeepWare: Imaging performance counters with deep learning to detect ransomware. *IEEE Transactions on Computers*, 1.

Oliva, A., & Torralba, A. (2001). Modeling the shape of the scene: A holistic representation of the spatial envelope. *International Journal of Computer Vision*, *42*(3), 145–175.

Onwuzurike, L., Mariconti, E., Andriotis, P., Cristofaro, E. D., Ross, G., & Stringhini, G. (2019). Mamadroid: Detecting android malware by building Markov chains of behavioral models (extended version). *ACM Transactions on Privacy and Security*, *22*(2), 1–34.

Or-Meir, O., Cohen, A., Elovici, Y., Rokach, L., & Nissim, N. (2021). Pay attention: Improving classification of PE malware using attention mechanisms based on system call analysis. In *2021 international joint conference on neural networks* (pp. 1–8).

Pagliardini, M., Gupta, P., & Jaggi, M. (2017). Unsupervised learning of sentence embeddings using compositional n-gram features. arXiv preprint arXiv:1703.02507.

Pintor, M., Demetrio, L., Sotgiu, A., Demontis, A., Carlini, N., Biggio, B., et al. (2022). Indicators of attack failure: Debugging and improving optimization of adversarial examples. *Advances in Neural Information Processing Systems*, *35*, 23063–23076.

Qiao, Y., Zhang, W., Tian, Z., Yang, L. T., Liu, Y., & Alazab, M. (2022). Adversarial ELF malware detection method using model interpretation. *IEEE Transactions on Industrial Informatics*, *19*(1), 605–615.

Qiao, Y., Zhang, B., & Zhang, W. (2020). Malware classification method based on word vector of bytes and multilayer perception. In *ICC 2020-2020 IEEE international conference on communications* (pp. 1–6). IEEE.

Rao, D., & Mane, S. (2021). Zero-shot learning approach to adaptive cybersecurity using explainable AI. arXiv preprint arXiv:2106.14647.

Ren, S., Zhou, L., Liu, S., Wei, F., Zhou, M., & Ma, S. (2021). Semface: Pre-training encoder and decoder with a semantic interface for neural machine translation. In *Proceedings of the 59th annual meeting of the association for computational linguistics and the 11th international joint conference on natural language processing (volume 1: long papers)* (pp. 4518–4527).

Rezende, E., Ruppert, G., Carvalho, T., Ramos, F., & De Geus, P. (2017). Malicious software classification using transfer learning of resnet-50 deep neural network. In *2017 16th IEEE international conference on machine learning and applications* (pp. 1011–1014). IEEE.

Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). " Why should i trust you?" Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 1135–1144).

Rublee, E., Rabaud, V., Konolige, K., & Bradski, G. (2011). ORB: An efficient alternative to SIFT or SURF. In *2011 international conference on computer vision* (pp. 2564–2571).

Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., et al. (2015). Imagenet large scale visual recognition challenge. *International Journal of Computer Vision*, *115*(3), 211–252.

Sabour, S., Frosst, N., & Hinton, G. E. (2017a). Dynamic routing between capsules. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, & R. Garnett (Eds.), *Vol. 30*, *Advances in neural information processing systems*. Curran Associates, Inc..

Sabour, S., Frosst, N., & Hinton, G. E. (2017b). Dynamic routing between capsules. arXiv preprint arXiv:1710.09829.

Sasi, S. B., & Sivanandam, N. (2015). A survey on cryptography using optimization algorithms in WSNs. *Indian Journal of Science and Technology*, *8*(3), 216.

Sharmeen, S., Ahmed, Y. A., Huda, S., Koçer, B. Ş., & Hassan, M. M. (2020). Avoiding future digital extortion through robust protection against ransomware threats using deep learning based adaptive approaches. *IEEE Access*, *8*, 24522–24534.

Shrikumar, A., Greenside, P., & Kundaje, A. (2017). Learning important features through propagating activation differences. In *International conference on machine learning* (pp. 3145–3153). PMLR.

Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556.

Sudhakar, & Kumar, S. (2021). MCFT-CNN: Malware classification with fine-tune convolution neural networks using traditional and transfer learning in internet of things. *Future Generation Computer Systems*, *125*, 334–351.

Swain, M. J., & Ballard, D. H. (1991). Color indexing. *International Journal of Computer Vision*, *7*(1), 11–32.

Szegedy, C., Ioffe, S., Vanhoucke, V., & Alemi, A. A. (2017). Inception-v4, inception-resnet and the impact of residual connections on learning. In *Thirty-first AAAI conference on artificial intelligence*.

Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., et al. (2015). Going deeper with convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 1–9).

Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2016). Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 2818–2826).

Tan, M., & Le, Q. (2019). Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning* (pp. 6105–6114). PMLR.

Tobiyama, S., Yamaguchi, Y., Shimada, H., Ikuse, T., & Yagi, T. (2016). Malware detection with deep neural network using process behavior. *Vol. 2*, In *2016 IEEE 40th annual computer software and applications conference* (pp. 577–582).

Vasan, D., Alazab, M., Wassan, S., Naeem, H., Safaei, B., & Zheng, Q. (2020). IMCFN: Image-based malware classification using fine-tuned convolutional neural network architecture. *Computer Networks*, *171*, Article 107138.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., et al. (2017). Attention is all you need. In *Advances in neural information processing systems*, (pp. 5998–6008).

Vivek, Y., Ravi, V., Mane, A. A., & Naidu, L. R. (2022). Explainable artificial intelligence and causal inference based ATM fraud detection. arXiv preprint arXiv:2211.10595.

Xiao, M., Guo, C., Shen, G., Cui, Y., & Jiang, C. (2021). Image-based malware classification using section distribution information. *Computers & Security*, *110*, Article 102420.

Xu, G., Xin, G., Jiao, L., Liu, J., Liu, S., Feng, M., et al. (2023). Ofei: A semi-black-box android adversarial sample attack framework against dlaas. *IEEE Transactions on Computers*.

Yakura, H., Shinozaki, S., Nishimura, R., Oyama, Y., & Sakuma, J. (2019). Neural malware analysis with attention mechanism. *Computers & Security*, *87*, Article 101592.

Ye, R., & Dai, Q. (2021). Implementing transfer learning across different datasets for time series forecasting. *Pattern Recognition*, *109*, Article 107617.

Yuan, B., Wang, J., Liu, D., Guo, W., Wu, P., & Bao, X. (2020). Byte-level malware classification based on Markov images and deep learning. *Computers & Security*, *92*, Article 101740.

Zhao, Z., Li, Z., Zhang, F., Yang, Z., Luo, S., Li, T., et al. (2023). SAGE: Steering the adversarial generation of examples with accelerations. *IEEE Transactions on Information Forensics and Security*, *18*, 789–803.

Zhong, F., Cheng, X., Yu, D., Gong, B., Song, S., & Yu, J. (2023). Malfox: camouflaged adversarial malware example generation based on conv-GANs against black-box detectors. *IEEE Transactions on Computers*.

Zhu, J., Jang-Jaccard, J., Singh, A., Watters, P. A., & Camtepe, S. (2021). Task-aware meta learning-based siamese neural network for classifying obfuscated malware. arXiv preprint arXiv:2110.13409.